INGENIERÍA DE SOFTWARE · PATRONES DE SOFTWARE

Caso 2 — Planificación de estiba en una terminal fluvial de carga

Abstract Factory | Factory Method | Adapter | Bridge | Facade

Duración: 2 horas **Lenguaje:** Java (SE 17+) **Modalidad:** individual, aplicación de consola

1. Contexto

La **Terminal Fluvial de Barrancabermeja** opera tres muelles especializados sobre el río Magdalena y despacha convoyes de barcazas hacia Barranquilla. Antes de cada zarpe, el *planeador de operaciones* debe producir un **Plan de Estiba**: el documento que indica qué carga va en cada bodega de la barcaza, en qué orden se carga, con qué equipo y en cuánto tiempo.

El problema operativo es que cada muelle es un mundo distinto. El muelle de granel sólido usa bandas transportadoras y su documento de embarque exige certificado de humedad; el de contenedores usa grúa pórtico y trabaja con listas de unidades sellado por sellado; el de líquidos usa brazos de carga y exige certificado de compatibilidad química del tanque anterior. Cuando el equipo de sistemas intentó unificar todo en una sola clase con condicionales, terminó con un método de 400 líneas que nadie se atreve a tocar y que ya provocó un plan de estiba con un desbalance de peso que obligó a reacomodar 600 toneladas en pleno cargue.

Además, en aguas bajas se aprobó un plan por calado sin consultar el nivel real del río y la barcaza varó seis horas: el nivel se leía a mano de dos instrumentos de distinto fabricante. Y el tiempo de navegación hasta Barranquilla cambia según el régimen del río y el remolcador disponible, y hoy se calcula en una hoja aparte. El equipo de operaciones quiere una única entrada para todo el proceso.

2. Reglas de negocio

2.1 Tipos de terminal (familias de elementos)

Cada terminal define de forma **inseparable** tres elementos: el **equipo de cargue**, el **validador de estabilidad** y el **formato del documento de embarque**.

Terminal	Equipo de cargue	Tasa	Validador de estabilidad	Documento de embarque
GRANEL_SOLIDO	Banda transportadora BT-3	320 t/h	Desbalance máx. 8 %, calado máx. 1.80 m	Conocimiento de embarque con certificado de humedad
CONTENEDORES	Grúa pórtico móvil GP-1	180 t/h	Desbalance máx. 5 %, calado máx. 2.10 m	Lista de unidades con número de sello por contenedor
LIQUIDOS	Brazo de carga BC-2	240 t/h	Desbalance máx. 3 %, calado máx. 1.95 m	Manifiesto con certificado de compatibilidad química

2.2 Unidades de carga y su registro desde el manifiesto

El manifiesto llega como una lista de líneas de texto con formato distinto según el tipo de carga. Cada tipo de unidad calcula su peso y su factor de acomodación de manera diferente:

Tipo	Línea del manifiesto	Cálculo del peso bruto (t)
Contenedor	`CNT;MSKU1234567;40;18.5;SELLO-8891` (sigla; matrícula; pies; carga neta t; sello)	carga neta + tara (20 pies = 2.2 t, 40 pies = 3.8 t)
Lote de granel	`GRA;CARBON-TERMICO;850;13.5` (sigla; producto; toneladas; % humedad)	toneladas declaradas × (1 + %humedad/100)
Tanque líquido	`LIQ;ACEITE-PALMA;120000;0.915` (sigla; producto; litros; densidad kg/L)	litros × densidad / 1000

2.3 Bodegas de la barcaza

Cada barcaza tiene cuatro bodegas: **B1** y **B2** a proa, **B3** y **B4** a popa. Capacidades: B1 = 600 t, B2 = 750 t, B3 = 750 t, B4 = 600 t.

2.4 Regla de balance

Sea `Pproa = peso(B1) + peso(B2)` y `Ppopa = peso(B3) + peso(B4)`. El desbalance se calcula como:

`desbalance % = |Pproa − Ppopa| / pesoTotal × 100`

Si el desbalance supera el máximo permitido por el validador de la terminal, el plan se marca como **NO APROBADO** e indica cuántas toneladas habría que trasladar del lado pesado al liviano para quedar justo en el límite.

2.5 Calado estimado

`calado (m) = 0.55 + (pesoTotal / 4200)`. Si supera el máximo del validador, el plan se marca **NO APROBADO** por calado.

2.6 Nivel del río y calado efectivo

El nivel del canal se lee con dos instrumentos de terceros que no se pueden modificar:

- `LimnimetroRH100` expone `double nivelMetros()`.
- `SensorRadarSR7` expone `String lectura()` con el formato `NIVEL_CM=<cm>;ESTADO=<OK|FALLA>`. Si el estado es `FALLA` la lectura no se puede usar.
- El resto del sistema solo conoce la interfaz `MedidorNivelRio` con `double nivelMetros()`.

`calado disponible (m) = nivel del canal − 0.40` (franco de seguridad). El **calado máximo efectivo** es el menor entre el máximo del validador de la terminal y el calado disponible; la regla 2.5 se aplica con ese valor. En la demostración el limnímetro marca **2.90 m** y el radar devuelve `NIVEL_CM=145;ESTADO=OK`.

2.7 Travesía Barrancabermeja – Barranquilla

La distancia es de **580 km**. El régimen del río y el remolcador varían de forma independiente: hoy hay 3 regímenes y 2 remolcadores, y para 2027 se compra un tercer remolcador.

Régimen	Factor de velocidad	Horas de navegación por día
AGUAS_NORMALES	1.00	24
AGUAS_BAJAS	0.75	12 (solo diurna)
CRECIENTE	0.85	24

Remolcador	Velocidad base	Peso máximo que puede empujar
RIO-1800 (1.800 HP)	10 km/h	2.000 t
RIO-3200 (3.200 HP)	13 km/h	5.000 t

`horas = distancia / (velocidad base × factor del régimen)`; `días de navegación = horas / horas por día`, redondeado hacia arriba; la llegada es la fecha de zarpe más esos días. Si el peso total del plan supera lo que el remolcador puede empujar, la travesía no se puede planificar.

3. Requerimiento

Construya en Java una aplicación de consola que, dado un tipo de terminal y un manifiesto de carga, produzca un plan de estiba completo, balanceado y verificado, con su calado según el nivel real del río y la planificación de su travesía.

4. Patrones obligatorios (exactamente cinco)

> **Abstract Factory — familias por tipo de terminal**
>
> Defina `FabricaTerminal` con `crearEquipoDeCargue()`, `crearValidadorEstabilidad()` y `crearDocumentoEmbarque()`, con las tres implementaciones concretas de la tabla 2.1. El planeador debe recibir la fábrica y trabajar solo con las interfaces de producto.

> **Factory Method — creación de unidades de carga desde el manifiesto**
>
> Defina una clase abstracta `RegistradorDeManifiesto` con un **método plantilla** público `procesar(List<String> lineas)` que recorre, valida el formato y acumula, y un **método fábrica** `protected abstract UnidadDeCarga crearUnidad(String linea)`. Implemente `RegistradorContenedor`, `RegistradorGranel` y `RegistradorLiquido`, cada uno con su propio parseo y su propia fórmula de peso (tabla 2.2). Las líneas cuyo formato no corresponda al registrador deben reportarse como rechazadas, sin detener el proceso.

> **Adapter — instrumentos de nivel del río**
>
> Escriba un adaptador por cada instrumento de terceros (`LimnimetroRH100`, `SensorRadarSR7`) hacia `MedidorNivelRio`. El del radar debe interpretar la cadena, convertir centímetros a metros y lanzar `IllegalStateException` si el estado es `FALLA`. Ninguna clase fuera de los adaptadores puede conocer los instrumentos.

> **Bridge — régimen del río × remolcador**
>
> `Travesia` (abstracción) mantiene una referencia a `Remolcador` (implementación). `TravesiaAguasNormales`, `TravesiaAguasBajas` y `TravesiaCreciente` definen el factor de velocidad y las horas de navegación por día; `RemolcadorRio1800` y `RemolcadorRio3200` definen velocidad base y peso máximo. `planificar(double pesoTotal, LocalDate zarpe)` combina ambas jerarquías. Agregar un tercer remolcador no debe requerir tocar ningún régimen, y no se admiten clases como `BajasConRio3200`.

> **Facade — central de la terminal**
>
> `CentralTerminal` ofrece operaciones de alto nivel: `procesarManifiesto(...)`, `generarPlan(...)`, `evaluarNivelDelRio(...)` y `planificarTravesia(...)`. Internamente coordina la fábrica de terminal, los registradores, los adaptadores de nivel y los remolcadores. La clase `Main` solo habla con la fachada: **no** puede instanciar fábricas de terminal, registradores, adaptadores ni remolcadores.

5. Componente algorítmico obligatorio

1. **Distribución de carga (heurística First Fit Decreasing):** ordene las unidades por peso descendente y asigne cada una a la primera bodega, en orden B1, B2, B3, B4, que tenga capacidad disponible suficiente. Si ninguna la admite, regístrela en una lista de carga no embarcada.
2. **Balance:** aplique la regla 2.4 e informe el porcentaje con dos decimales y las toneladas a trasladar si aplica.
3. **Calado:** aplique la regla 2.5 con dos decimales.
4. **Tiempo de cargue:** horas = `pesoTotal / tasa del equipo`, expresado en horas y minutos (p. ej. 4 h 37 min).
5. **Secuencia de cargue:** genere la secuencia bodega por bodega, y dentro de cada bodega de mayor a menor peso, numerando cada movimiento.
6. **Ocupación:** porcentaje de utilización de cada bodega con un decimal.

Los cálculos siguientes se resuelven dentro de las clases de sus patrones:

7. **Calado efectivo:** nivel del río, calado disponible y máximo efectivo (regla 2.6); reevaluación del plan con ese máximo y, si no aprueba, las toneladas que habría que descargar.
8. **Travesía:** horas y minutos de navegación (redondeados al minuto), días y fecha de llegada.

6. Programa de demostración

1. Procese un manifiesto de al menos 10 líneas de contenedores, incluyendo dos líneas con formato inválido que deben quedar reportadas como rechazadas.
2. Genere el plan de estiba completo para la terminal de CONTENEDORES e imprímalo.
3. Repita el proceso con un manifiesto de granel sólido cuyo peso total provoque un desbalance por encima del límite, para evidenciar el estado NO APROBADO y las toneladas a trasladar.

4. Lea el nivel del río con ambos instrumentos y reevalúe el plan de contenedores con el nivel del radar (aguas bajas).
5. Planifique la travesía del plan de contenedores, con zarpe el 2026-05-12: intente con el RIO-1800 dentro de `try/catch` y luego use el RIO-3200 en los tres regímenes.

Formato sugerido de salida

```
=== TERMINAL FLUVIAL DE BARRANCABERMEJA ===
Registrador: CONTENEDORES | lineas leidas: 12 | aceptadas: 10 | rechazadas: 2
  [RECHAZADA] GRA;CARBON;850;13.5   -> no corresponde al registrador de contenedores
  [RECHAZADA] CNT;MSKU999;XX;18.5   -> pies no numerico

PLAN DE ESTIBA No. PE-2026-0148
Barcaza: BZ-4417   Zarpe: 2026-05-12   Terminal: CONTENEDORES
Equipo: Grua portico movil GP-1 (180 t/h)

BODEGA   CARGA (t)   CAPACIDAD   OCUPACION   UNIDADES
B1          578.30       600.0      96.4 %   3
B2          731.10       750.0      97.5 %   4
B3          690.40       750.0      92.1 %   2
B4          512.00       600.0      85.3 %   1
TOTAL      2511.80

Proa: 1309.40 t | Popa: 1202.40 t | Desbalance: 4.26 % (max 5.00 %)     -> OK
Calado estimado: 1.15 m (max 2.10 m) -> OK
Tiempo estimado de cargue: 13 h 57 min
ESTADO DEL PLAN: APROBADO

SECUENCIA DE CARGUE
 1. B1 <- MSKU1234567 (40 pies) 22.30 t
 2. B1 <- TCLU8877213 (40 pies) 21.90 t
 ...
DOCUMENTO: Lista de unidades con numero de sello por contenedor

--- NIVEL DEL RIO (adaptadores) ---
Limnimetro RH-100 -> 2.90 m | calado disponible 2.50 m | calado efectivo max 2.10 m
Radar SR-7 "NIVEL_CM=145;ESTADO=OK" -> 1.45 m | calado disponible 1.05 m | calado efectivo max 1.05 m
PE-2026-0148 reevaluado con aguas bajas: calado 1.15 m (max 1.05 m) -> NO APROBADO por calado
  Toneladas a descargar para quedar en el limite: 411.80 t

--- TRAVESIA PE-2026-0148 | 2511.80 t | 580 km | zarpe 2026-05-12 ---
[ERROR CONTROLADO] El remolcador RIO-1800 no puede empujar 2511.80 t (max 2000 t)
REMOLCADOR  REGIMEN         NAVEGACION   DIAS   LLEGADA
RIO-3200    AGUAS_NORMALES  44 h 37 min     2   2026-05-14
RIO-3200    AGUAS_BAJAS     59 h 29 min     5   2026-05-17
RIO-3200    CRECIENTE       52 h 29 min     3   2026-05-15
```

7. Restricciones y entregables

- Java estándar, sin frameworks ni librerías externas. Sí puede usar `java.util`, `java.time` y `String.split`.
- Las clases `LimnimetroRH100` y `SensorRadarSR7` van en su propio paquete `terceros` y se tratan como código que no se puede editar.
- Paquetes sugeridos: `terceros`, `terminal`, `manifiesto`, `plan`, `rio`, `travesia`, `fachada`, `app`.
- Entregue el proyecto comprimido (.zip) con código compilable y captura de la salida de consola.
- Incluya `PATRONES.md` (máximo una página) explicando dónde quedó cada patrón y qué problema del caso resuelve.

Fuera de alcance

Interfaz gráfica, lectura de archivos reales (el manifiesto va quemado en una lista dentro del código), persistencia, pruebas automatizadas, optimización exacta del empaquetado (se exige la heurística indicada, no el óptimo) y conexión real con los instrumentos de nivel.